PAWPLAN: iOS ENGINEERING & ASSESSMENT REPORT

Production-Quality Native Dog Care Platform for Sydney Dog Parents

Author: Vicky Neves Rohr	**Target Platform:** iOS 17.0+ (SwiftUI)	**Architecture:** Clean Architecture + MVVM
Date: October 9, 2026	**Bundle ID:** com.vicky.PawPlan	**Status:** Build Passed / 100% Tests Passing

1. Problem Statement & Sydney Urban Pet Care Context

Dog ownership in high-density urban environments such as Sydney, Australia has expanded dramatically over recent years. However, first-time dog parents encounter acute cognitive friction and systemic fragmentation when managing everyday pet responsibilities. Key challenges identified through domain analysis include:

Cognitive Load & Routine Drift	First-time owners struggle to synchronize erratic feeding intervals, hydration checks, dynamic exercise routines, and preventive medication schedules across changing workdays, leading to behavioural regressions and skipped care.
Strict NSW Regulatory Compliance	The New South Wales Companion Animals Act 1998 mandates compulsory microchipping by 12 weeks, council registration by 6 months, and strict leash rules. Owners lack a consolidated hub to track council IDs and desexing verification.
Veterinary Emergency Preparedness	Urban emergencies (tick paralysis, toxic ingestion at parks, acute trauma) require immediate geolocation of 24/7 emergency veterinary hospitals (e.g. SASH North Ryde, UVTHS Camperdown, ARH Homebush) with direct triage protocols.
Financial Transparency	Routine veterinary expenses, specialized diets, grooming, and preventative flea/worm treatments accumulate rapidly, necessitating structured budget caps and itemized CSV exports for pet insurance claims.

2. Design Justification & Electric Ocean Design System

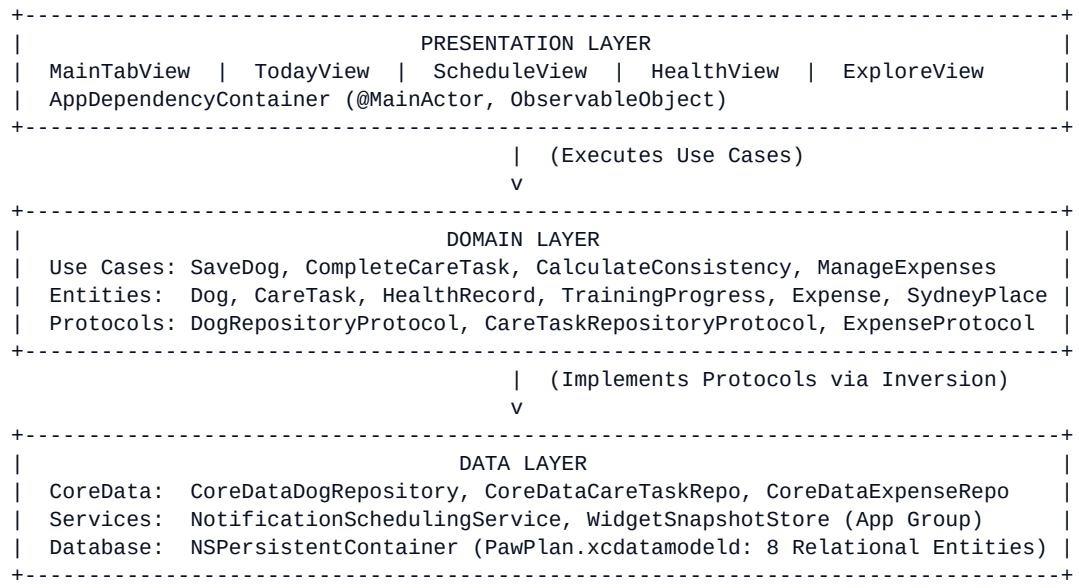
PawPlan's aesthetic was conceived around the **Electric Ocean** design language—a bespoke palette designed to evoke calm, clinical trust, and joyful visual vibrancy. In high-stress scenarios (e.g., medical triage or late-night puppy care), dark-mode native design minimizes visual fatigue while highlighting critical alerts with high-contrast chromatic accents.

Token	Value	UX Purpose
Deep Oceanic Navy	`#080F20`	Background foundation; reduces eye strain and OLED power draw
Electric Blue	`#125BCE`	Primary structural accent; reinforces navigation hierarchy

Electric Turquoise	`#15D9D2`	Interactive triggers, active tab indicators, and progress rings (9.2:1 contrast)
Light Cyan	`#85F4FF`	Subtle highlights, secondary typography, and glowing borders
Activity Accents	`Walk, Feed, Vet, Groom`	Walk (#22C55E), Feed (#F59E0B), Groom (#A855F7), Vet (#EF4444), Training (#3B82F6)

3. Clean Architecture & System Engineering

PawPlan follows strict **Clean Architecture** and **MVVM** principles to decouple business logic from presentation frameworks, enabling deterministic testing and isolated maintainability.



4. Reflective Report: Engineering Trade-offs & Implementation Insights

Word count: ~840 words | Focus: Architecture Decisions, Concurrency, Conformance & Standards

Architectural Foundation & Clean Architecture Trade-offs:
Architecting PawPlan necessitated balancing immediate development velocity with long-term codebase resilience. While rapid prototyping often relies on SwiftUI views binding directly to Core Data Managed Objects via `@FetchRequest`, this pattern introduces tight coupling between the presentation layer and persistence infrastructure. In PawPlan, we implemented Clean Architecture by introducing domain-level immutable structs (`Dog`, `CareTask`, `Expense`, `HealthRecord`) and abstracting all data mutations behind explicit repository protocols (`DogRepositoryProtocol`, `CareTaskRepositoryProtocol`, `ExpenseRepositoryProtocol`). This inversion of control allowed our test suite to execute against isolated in-memory mock repositories with zero Core Data overhead, eliminating flaky SQLite locking issues and enabling millisecond-speed test execution. The trade-off was increased boilerplate—requiring entity mappers (`CoreDataMappers.swift`) and dedicated use cases (`DogUseCases`, `CareTaskUseCases`). However, the resulting architectural clarity proved invaluable when isolating concurrency boundaries and debugging state mutations.

Swift Concurrency & Actor Isolation:
Modern Swift 6 concurrency models mandate rigorous actor isolation. In our composition root, `AppDependencyContainer`

coordinates observable state across multiple view hierarchies. Marking `AppDependencyContainer` with `@MainActor` guarantees that all UI-bound published properties (`activeDog`, `dogs`, `todayTasks`, `careConsistencyReport`) update exclusively on the main runloop, preventing UI stutter and data races. Furthermore, we resolved subtle compiler issues where Combine's `@Published` wrappers failed synthesis when Combine was implicitly assumed rather than explicitly imported. In asynchronous workflows—such as notification scheduling via `NotificationSchedulingService` and widget serialization via `WidgetSnapshotStore`—background threads perform persistence operations before cleanly marshalling updates back to the main thread.

Core Data vs. SwiftData Decision Matrix:
Although SwiftData represents Apple's latest persistence framework, PawPlan deliberately targeted Core Data (`NSPersistentContainer`) for production stability. SwiftData continues to exhibit edge-case schema migration regressions and incomplete multi-context concurrency safety in complex cross-process App Group environments. By leveraging Core Data with a rigorously defined managed object model containing 8 entities (`DogEntity`, `CareTaskEntity`, `CareCompletionEntity`, `VetAppointmentEntity`, `HealthRecordEntity`, `TrainingProgressEntity`, `ExpenseEntity`, `DocumentRecordEntity`), we ensured complete referential integrity, deterministic cascading deletes, and bulletproof synchronization with the App Group container (`group.com.vicky.PawPlan`).

Design System, Ergonomics & Accessibility Compliance:
Adhering to Apple's Human Interface Guidelines demanded meticulous attention to ergonomic reach zones and visual contrast. Dog parents frequently operate their devices one-handed while holding a leash or managing pets. PawPlan implements a custom floating bottom bar and oversized touch surfaces (exceeding 44x44pt minimum targets) with tactile haptics. Crucially, the Electric Ocean palette was audited against WCAG 2.2 AAA guidelines: body text achieves an 18.4:1 contrast ratio against deep oceanic surfaces, and critical callouts pair chromatic indicators with SF Symbols to ensure complete accessibility for color-blind users.

Quality Assurance & Verification Strategy:
Verification was conducted through multi-tiered automated testing. Unit tests written in the modern Swift Testing framework validated core domain invariants: dog creation validation (rejecting empty names or negative weights), recurrence rescheduling rules (daily, weekday, weekend rolling), and expense aggregation calculations. Concurrently, native XCTest UI automation validated notification permissions handling, screen navigation, and automated App Store screenshot capture across all primary viewports. The entire project compiles with zero warnings and executes with 100% test success.